

3.3.1.- Cursores explícitos.

Cuando una consulta devuelve múltiples filas, debemos declarar explícitamente un cursor para procesar las filas devueltas. Cuando declaramos un cursor, lo que hacemos es darle un nombre y asociarle una consulta usando la siguiente sintaxis:

```
CURSOR nombre_cursor [(parametro [, parametro] ...)] [RETURN tipo_devuelto] IS sentencia_select;
```

Donde **tipo_devuelto** debe representar un registro o una fila de una tabla de la base de datos, y **parámetro** sigue la siguiente sintaxis:

```
parametro := nombre_parametro [IN] tipo_dato [{:= | DEFAULT} expresion]
```

Ejemplos:

```
CURSOR cAgentes IS SELECT * FROM agentes;  
CURSOR cFamilias RETURN familias%ROWTYPE IS SELECT * FROM familias WHERE ...
```

Además, como hemos visto en la declaración, un cursor puede tomar parámetros, los cuales pueden aparecer en la consulta asociada como si fuesen constantes. Los parámetros serán de entrada, un cursor no puede devolver valores en los parámetros actuales. A un parámetro de un cursor no podemos imponerle la restricción **NOT NULL**.

```
CURSOR c1 (cat INTEGER DEFAULT 0) IS SELECT * FROM agentes WHERE categoria = cat;
```

Cuando abrimos un cursor, lo que se hace es ejecutar la consulta asociada e identificar el conjunto resultado, que serán todas las filas que emparejen con el criterio de búsqueda de la consulta. Para abrir un cursor usamos la sintaxis:

```
OPEN nombre_cursor [(parametro [, parametro] ...)];
```

Ejemplos:

```
OPEN cAgentes;  
OPEN c1(1);  
OPEN c1;
```

La sentencia **FETCH** devuelve una fila del conjunto resultado. Después de cada **FETCH**, el cursor avanza a la próxima fila en el conjunto resultado.

```
FETCH cFamilias INTO mi_id, mi_nom, mi_fam, mi_ofi;
```

Para cada valor de columna devuelto por la consulta **SELECT** asociada al cursor, debe haber una variable que se corresponda en la lista de variables después del **INTO**.

Para procesar un cursor entero deberemos hacerlo por medio de un bucle.

Hay varias formas de hacerlo. En el formato del ejemplo siguiente es necesario abrir el cursor previamente y tras recorrerlo cerrarlo.

```
BEGIN
...
OPEN cFamilias;
LOOP
    FETCH cFamilias INTO mi_id, mi_nom, mi_fam, mi_ofi;
    EXIT WHEN cFamilias%NOTFOUND;
    ...
END LOOP;
CLOSE cFamilias;
...
END;
```

Una vez cerrado el cursor podemos reabrirlo, pero cualquier otra operación que hagamos con el cursor cerrado lanzará la excepción **INVALID_CURSOR**.

Otra forma más sencilla es utilizar los bucles para cursores, los cuales declaran implícitamente una variable índice definida como **%ROWTYPE** para el cursor, abren el cursor, extraen los valores de cada fila del cursor, almacenándolas en la variable índice, y finalmente cierran el cursor.

```
BEGIN
...
FOR cFamilias_rec IN cFamilias LOOP
    --Procesamos las filas accediendo a
    --cFamilias_rec.identificador, cFamilias_rec.nombre,
    --cFamilias_rec.familia, ...
END LOOP;
...
END;
```

Autoevaluación

En PL/SQL los cursores son abiertos al definirlos.

- ☐ Verdadero.
- ☐ Falso.

No, un cursor se debe abrir por medio de la sentencia **OPEN**.

Efectivamente, debemos abrirlos por medio de la sentencia **OPEN**.

Solución

1. Incorrecto
2. Opción correcta

3.3.2.- Cursores variables.

Oracle, además de los cursores vistos anteriormente, nos permite definir cursores variables que son como punteros a cursores, por lo que podemos usarlos para referirnos a cualquier tipo de consulta. Los cursores serían estáticos y los cursores variables serían dinámicos.



[ITE \(Abraham Pérez Pérez\)](#) (Uso educativo no)

Para declarar un cursor variable debemos seguir 2 pasos:

- ✓ Definir un tipo **REF CURSOR** y entonces declarar una variable de ese tipo.

```
TYPE tipo_cursor IS REF CURSOR RETURN agentes%ROWTYPE;  
cAgentes tipo_cursor;
```

- ✓ Una vez definido el cursor variable debemos asociarlo a una consulta (notar que esto no se hace en la parte declarativa, sino dinámicamente en la parte de ejecución) y esto lo hacemos con la sentencia **OPEN-FOR** utilizando la siguiente sintaxis:

```
OPEN nombre_variable_cursor FOR sentencia_select;  
OPEN cAgentes FOR SELECT * FROM agentes WHERE oficina = 1;
```

Un cursor variable no puede tomar parámetros. Podemos usar los atributos de los cursores para cursores variables.

Además, podemos usar varios **OPEN-FOR** para abrir el mismo cursor variable para diferentes consultas. No necesitamos cerrarlo antes de reabrirlo. Cuando abrimos un cursor variable para una consulta diferente, la consulta previa se pierde.

Una vez abierto el cursor variable, su manejo es idéntico a un cursor. Usaremos **FETCH** para traernos las filas, usaremos sus atributos para hacer comprobaciones y lo cerraremos cuando dejemos de usarlo.

```
DECLARE  
TYPE cursor_Agentes IS REF CURSOR RETURN agentes%ROWTYPE;  
cAgentes cursor_Agentes;  
agente cAgentes%ROWTYPE;  
BEGIN  
...  
OPEN cAgentes FOR SELECT * FROM agentes WHERE oficina = 1;  
LOOP  
    FETCH cAgentes INTO agente;  
    EXIT WHEN cAgentes%NOTFOUND;  
    ...  
END LOOP;  
CLOSE cAgentes;  
...  
END;
```


También puede utilizarse el bucle **FOR** con cursores variable definidos dentro del mismo bucle:

```
for va_cursor in (select ...) loop
...
end loop;
```

Autoevaluación

A los cursores variables no podemos pasarles parámetros al abrirlos.

- ☐ Verdadero.
- ☐ Falso.

Correcto, veo que lo vas entendiendo.

No, a este tipo de cursores no podemos pasarles parámetros al abrirlos.

Solución

1. Opción correcta
2. Incorrecto

Los cursores variables se abren exactamente igual que los cursores explícitos.

- ☐ Verdadero.
- ☐ Falso.

No, al abrir un cursor variable debemos asociarle la consulta por medio de la sentencia **OPEN-FOR**.

Correcto, ya que debemos abrirlo por medio de la sentencia **OPEN-FOR** con la que le asociamos la consulta.

Solución

1. Incorrecto
2. Opción correcta

4.- Abstracción en PL/SQL.

Caso práctico

María, gracias a la ayuda de **Juan**, tiene bastante claro cómo programar en PL/SQL pero no entiende muy bien cómo integrar todo esto con la base de datos de juegos on-line. Sabe que puede utilizar unos tipos de datos, que hay unas estructuras de control y que se pueden manejar los errores que surjan, pero lo que no sabe es cómo y donde utilizar todo eso.

Juan le explica que lo que hasta ahora ha aprendido es el comienzo, pero que ahora viene lo bueno y que será donde le va a encontrar pleno sentido a lo aprendido anteriormente. Le explica que PL/SQL permite crear funciones y procedimientos y además agruparlos en paquetes y que eso será lo que realmente van a hacer con la base de datos de juegos on-line. Deberán ver qué es lo que utilizan más comúnmente e implementarlo en PL/SQL utilizando funciones y procedimientos según convenga. **Juan** la tranquiliza y le dice que lo primero que va a hacer es explicarle cómo se escriben dichas funciones y procedimientos y luego pasarán a implementar alguno y que así verá la potencia real de PL/SQL. **María** se queda más tranquila y está deseando implementar esa primera función o procedimiento que le resolverá la gran duda que tiene.



[ITE](#) (Uso educativo nc)

Hoy día cualquier lenguaje de programación permite definir diferentes grados de abstracción en sus programas. La abstracción permite a los programadores crear unidades lógicas y posteriormente utilizarlas pensando en qué hace y no en cómo lo hace. La abstracción se consigue utilizando funciones, procedimientos, librerías, objetos, etc.

PL/SQL nos permite definir funciones y procedimientos. Además nos permite agrupar todas aquellas que tengan relación en paquetes. También permite la utilización de objetos. Todo esto es lo que veremos en este apartado y conseguiremos darle modularidad a nuestras aplicaciones, aumentar la reusabilidad y mantenimiento del código y añadir grados de abstracción a los problemas.

Para saber más

En los siguientes enlaces puedes ampliar información sobre la abstracción en programación.

[Abstracción en los lenguajes de programación.](#)

[Programación orientada a objetos.](#)

4.1.- Subprogramas.

Los **subprogramas** son bloques de código PLSQL, referenciados bajo un nombre, que realizan una acción determinada. Les podemos pasar parámetros y los podemos invocar. Además, los subprogramas pueden estar almacenados en la base de datos o estar encerrados en otros bloques. Si el programa está almacenado en la base de datos, podremos invocarlo si tenemos permisos suficientes y si está encerrado en otro bloque lo podremos invocar si tenemos visibilidad sobre el mismo.

Hay dos clases de subprogramas: las funciones y los procedimientos. Las funciones devuelven un valor y los procedimientos no.

Para declarar un subprograma utilizamos la siguiente sintaxis:

Sintaxis de Funciones.

```
FUNCTION nombre [(parametro [, parametro] ...)]  
    RETURN tipo_dato IS  
    [declaraciones_locales]  
BEGIN  
    sentencias_ejecutables  
[EXCEPTION  
    manejadores_de_excepciones]  
END [nombre];
```

Sintaxis de Procedimientos.

```
PROCEDURE nombre [( parametro [, parametro] ... )] IS  
    [declaraciones_locales]  
BEGIN  
    sentencias_ejecutables  
[EXCEPTION manejadores_de_excepciones]  
END [nombre];
```

Donde:

```
parametro := nombre_parametro [IN|OUT|IN OUT] tipo_dato [{:=|DEFAULT} expresion]
```

Algunas consideraciones que debes tener en cuenta son las siguientes:

- ✓ No podemos imponer una restricción **NOT NULL** a un parámetro.
- ✓ No podemos especificar una restricción del tipo:

```
PROCEDURE KK(a NUMBER(10)) IS ...           --ilegal
```

- ✓ Una función siempre debe acabar con la sentencia **RETURN**.

Podemos definir subprogramas al final de la parte declarativa de cualquier bloque. En Oracle, cualquier identificador debe estar declarado antes de usarse, y eso mismo pasa con los subprogramas, por lo que deberemos declararlos antes de usarlos.

```
DECLARE
hijos NUMBER;
FUNCTION hijos_familia( id_familia NUMBER )
    RETURN NUMBER IS
    hijos NUMBER;
BEGIN
    SELECT COUNT(*) INTO hijos FROM agentes
        WHERE familia = id_familia;
    RETURN hijos;
END hijos_familia;
BEGIN
...
END;
```

Si quisiéramos definir subprogramas en orden alfabético o lógico, o necesitamos definir subprogramas mutuamente recursivos (uno llama a otro, y éste a su vez llama al anterior), deberemos usar la definición hacia delante, para evitar errores de compilación.

```
DECLARE
    PROCEDURE calculo(...);          --declaración hacia delante
    --Definimos subprogramas agrupados lógicamente
    PROCEDURE inicio(...) IS
    BEGIN
        ...
        calculo(...);
        ...
    END;
    ...
BEGIN
    ...
END;
```

Autoevaluación

Una función siempre debe devolver un valor.

- ☐ Verdadero.
- ☐ Falso.

Efectivamente, una función obligatoriamente debe devolver un valor.

Incorrecto, creo que debería repasar este apartado.

Solución

1. Opción correcta
2. Incorrecto

En PL/SQL no podemos definir subprogramas mutuamente recursivos.

- ☐ Verdadero.
- ☐ Falso.

Incorrecto ya que podemos hacerlo usando la definición hacia delante.

¡Muy bien!

Solución

1. Incorrecto
2. Opción correcta

4.1.1.- Almacenar subprogramas en la base de datos.

Para almacenar un subprograma en la base de datos utilizaremos la misma sintaxis que para declararlo, anteponiendo **CREATE [OR REPLACE]** a **PROCEDURE** o **FUNCTION**, y finalizando el subprograma con una línea que simplemente contendrá el carácter '/' para indicarle a Oracle que termina ahí. Si especificamos **OR REPLACE** y el subprograma ya existía, éste será reemplazado. Si no lo especificamos y el subprograma ya existe, Oracle nos devolverá un error indicando que el nombre ya está siendo utilizado por otro objeto de la base de datos.

```
CREATE OR REPLACE FUNCTION hijos_familia(id_familia NUMBER)
    RETURN NUMBER IS hijos NUMBER;
BEGIN
    SELECT COUNT(*) INTO hijos FROM agentes
        WHERE familia = id_familia;
RETURN hijos;
END;
/
```

Cuando los subprogramas son almacenados en la base de datos, para ellos no podemos utilizar las declaraciones hacia delante, por lo que cualquier subprograma almacenado en la base de datos deberá conocer todos los subprogramas que utilice.

Para invocar un subprograma usaremos la sintaxis:

```
nombre_procedimiento [(parametro [,parametro] ...)];
variable := nombre_funcion [(parametro [, parametro] ...)];
BEGIN
...
    hijos := hijos_familia(10);
...
END;
```

Si el subprograma está almacenado en la base de datos y queremos invocarlo desde SQL*Plus usaremos la sintaxis:

```
EXECUTE nombre_procedimiento [(parametros)];
EXECUTE :variable_sql := nombre_funcion [(parametros)];
```

Cuando almacenamos un subprograma en la base de datos éste es compilado antes. Si hay algún error se nos informará de los mismos y deberemos corregirlos creándolo de nuevo por medio de la cláusula **OR REPLACE**, antes de que el subprograma pueda ser utilizado.

Hay varias vistas del diccionario de datos que nos ayudan a llevar un control de los subprogramas, tanto para ver su código, como los errores de compilación. También hay algunos comandos de SQL*Plus que nos ayudan a hacer lo mismo pero de forma algo menos engorrosa.

El comando SQLPlus `show errors` tras la compilación de un procedimiento o función nos muestra los errores si los hay.

Vistas y comandos asociados a los subprogramas.

Información almacenada.	Vista del diccionario.	Comando.
Código fuente.	USER_SOURCE	DESCRIBE
Errores de compilación.	USER_ERRORS	SHOW ERRORS
Ocupación de memoria.	USER_OBJECT_SIZE	

También existe la vista `USER_OBJECTS` de la cual podemos obtener los nombres de todos los subprogramas almacenados.

Debes conocer

En la siguiente presentación te mostramos las vistas relacionadas con los subprogramas, la compilación de un subprograma con errores y cómo mostrar los mismos, la compilación de un subprograma correctamente y cómo mostrar su código fuente y la ejecución de un subprograma desde SQL*Plus y desde SQLDeveloper

[Presentación Subprogramas](#) (odp - 315,90 KB)
[Resumen textual alternativo](#)

Autoevaluación

Una vez que hemos almacenado un subprograma en la base de datos podemos consultar su código mediante la vista `USER_OBJECTS`.

- ☐ Verdadero.
- ☐ Falso.

No, podemos hacerlo mediante la vista `USER_SOURCE`.

Sí, lo estás captando a la primera.

Solución

1. Incorrecto
2. Opción correcta

4.1.2.- Parámetros de los subprogramas.

Ahora vamos a profundizar un poco más en los parámetros que aceptan los subprogramas y cómo se los podemos pasar a la hora de invocarlos.

Las variables pasadas como parámetros a un subprograma son llamadas **parámetros actuales**. Las variables referenciadas en la especificación del subprograma como parámetros, son llamadas **parámetros formales**.

Cuando llamamos a un subprograma, los parámetros actuales podemos escribirlos utilizando notación posicional o notación nombrada, es decir, la asociación entre parámetros actuales y formales podemos hacerla por posición o por nombre.

En la notación posicional, el primer parámetro actual se asocia con el primer parámetro formal, el segundo con el segundo, y así para el resto. En la notación nombrada usamos el operador => para asociar el parámetro actual al parámetro formal. También podemos usar notación mixta.

Los parámetros pueden ser de **entrada** al subprograma, de **salida**, o de **entrada/salida**. Por defecto si a un parámetro no le especificamos el modo, éste será de entrada. Si el parámetro es de salida o de entrada/salida, el parámetro actual debe ser una variable.

Un parámetro de entrada permite que pasemos valores al subprograma y no puede ser modificado en el cuerpo del subprograma. El parámetro actual pasado a un subprograma como parámetro formal de entrada puede ser una constante o una variable.

Un parámetro de salida permite devolver valores y dentro del subprograma actúa como variable no inicializada. El parámetro formal debe ser siempre una variable.

Un parámetro de entrada-salida se utiliza para pasar valores al subprograma y/o para recibirlos, por lo que un parámetro formal que actúe como parámetro actual de entrada-salida siempre deberá ser una variable.

Los parámetros de entrada los podemos inicializar a un valor por defecto. Si un subprograma tiene un parámetro inicializado con un valor por defecto, podemos invocarlo prescindiendo del parámetro y aceptando el valor por defecto o pasando el parámetro y sobrescribiendo el valor por defecto. Si queremos prescindir de un parámetro colocado entre medias de otros, deberemos usar notación nombrada o si los parámetros restantes también tienen valor por defecto, omitirlos todos.

Debes conocer

En el siguiente enlace puedes encontrar ejemplos sobre el paso de parámetros a nuestros subprogramas.

[Paso de parámetros a subprogramas.](#)

Autoevaluación

Indica de entre las siguientes afirmaciones las que creas que son correctas.

- ☐ En PL/SQL podemos usar la notación posicional para pasar parámetros.

- ☐ No existen los parámetros de salida ya que para eso existen las funciones.

- ☐ Los parámetros de entrada los podemos inicializar a un valor por defecto.

Mostrar retroalimentación

Solución

1. Correcto
2. Incorrecto
3. Correcto

4.1.3.- Sobrecarga de subprogramas y recursividad.

PL/SQL también nos ofrece la posibilidad de sobrecargar funciones o procedimientos, es decir, llamar con el mismo nombre subprogramas que realizan el mismo cometido y que aceptan distinto número y/o tipo de parámetros. No podemos sobrecargar subprogramas que aceptan el mismo número y tipo de parámetros y sólo difieren en el modo. Tampoco podemos sobrecargar subprogramas con el mismo número de parámetros y que los tipos de los parámetros sean diferentes, pero de la misma familia, o sean subtipos basados en la misma familia.



[ITE](#) (Uso educativo nc)

Debes conocer

En el siguiente enlace podrás ver un ejemplo de una función que es sobrecargada tres veces dependiendo del tipo de parámetros que acepta.

[Sobrecarga de subprogramas.](#)

PL/SQL también nos ofrece la posibilidad de utilizar la recursividad en nuestros subprogramas. Un subprograma es recursivo si éste se invoca a él mismo.

Debes conocer

En el siguiente enlace podrás ampliar información sobre la recursividad.

[Recursividad.](#)

En el siguiente enlace podrás ver un ejemplo del uso de la recursividad en nuestros subprogramas.

[Ejemplo del uso de la recursividad.](#)

Autoevaluación

En PL/SQL no podemos sobrecargar subprogramas que aceptan el mismo número y tipo de parámetros, pero sólo difieren en el modo.

- ☐ Verdadero.
- ☐ Falso.

Correcto, veo que lo has entendido.

No, no podemos hacerlo.

Solución

1. Opción correcta
2. Incorrecto

En PL/SQL no podemos utilizar la recursión y tenemos que imitarla mediante la iteración.

- ☐ Verdadero.
- ☐ Falso.

Incorrecto, creo que deberías mirar otra vez el segundo enlace del segundo "Debes conocer".

¡Muy bien!

Solución

1. Incorrecto
2. Opción correcta

4.2.- Paquetes.

Un paquete es un objeto que agrupa tipos, elementos y subprogramas. Suelen tener dos partes: la especificación y el cuerpo, aunque algunas veces el cuerpo no es necesario.

En la parte de especificación declararemos la interfaz del paquete con nuestra aplicación y en el cuerpo es donde implementaremos esa interfaz.



[ITE](#) (Uso educativo no)

Para crear un paquete usaremos la siguiente sintaxis:

```
CREATE [OR REPLACE] PACKAGE nombre AS
    [declaraciones públicas y especificación subprogramas]
END [nombre]
CREATE [OR REPLACE] PACKAGE BODY nombre AS
    [declaraciones privadas y cuerpo subprogramas especificados]
[BEGIN
    sentencias inicialización]
END [nombre];
```

La parte de inicialización sólo se ejecuta una vez, la primera vez que el paquete es referenciado.

Debes conocer

En el siguiente enlace te mostramos un ejemplo de un paquete que agrupa las principales tareas que realizamos con nuestra base de datos de ejemplo.

[Ejemplo de paquete.](#)

Para referenciar las partes visibles de un paquete, lo haremos por medio de la notación del punto.

```
BEGIN
    ...
    call_center.borra_agente( 10 );
    ...
END;
```

Para saber más

Oracle nos suministra varios paquetes para simplificar algunas tareas. En el siguiente enlace puedes encontrar más información sobre los mismos.

[Paquetes suministrados por Oracle.](#)

4.2.1.- Ejemplos de utilización del paquete DBMS_OUTPUT.

Oracle nos suministra un paquete público con el cual podemos enviar mensajes desde subprogramas almacenados, paquetes y disparadores, colocarlos en un buffer y leerlos desde otros subprogramas almacenados, paquetes o disparadores.

SQL*Plus permite visualizar los mensajes que hay en el buffer, por medio del comando **SET SERVEROUTPUT ON**. La utilización fundamental de este paquete es para la depuración de nuestros subprogramas.

Veamos uno a uno los subprogramas que nos suministra este paquete:

- ✓ Habilita las llamadas a los demás subprogramas. No es necesario cuando está activada la opción **SERVEROUTPUT**. Podemos pasarle un parámetro indicando el tamaño del buffer.

```
ENABLE
ENABLE( buffer_size IN INTEGER DEFAULT 2000);
```

- ✓ Deshabilita las llamadas a los demás subprogramas y purga el buffer. Como con **ENABLE** no es necesario si estamos usando la opción **SERVEROUTPUT**.

```
DISABLE
DISABLE();
```

- ✓ Coloca elementos en el buffer, los cuales son convertidos a **VARCHAR2**.

```
PUT
PUT(item IN NUMBER);
PUT(item IN VARCHAR2);
PUT(item IN DATE);
```

- ✓ Coloca elementos en el buffer y los termina con un salto de línea.

```
PUT_LINE
PUT_LINE(item IN NUMBER);
PUT_LINE(item IN VARCHAR2);
PUT_LINE(item IN DATE);
```

- ✓ Coloca un salto de línea en el buffer. Utilizado cuando componemos una línea usando varios **PUT**.

```
NEW_LINE
NEW_LINE();
```


- ✓ Lee una línea del buffer colocándola en el parámetro `line` y obviando el salto de línea. El parámetro `status` devolverá 0 si nos hemos traído alguna línea y 1 en caso contrario.

```
GET_LINE
GET_LINE(line OUT VARCHAR2, status OUT VARCHAR2);
```

- ✓ Intenta leer el número de líneas indicado en `numlines`. Una vez ejecutado, `numlines` contendrá el número de líneas que se ha traído. Las líneas traídas las coloca en el parámetro `lines` del tipo `CHARARR`, tipo definido el paquete `DBMS_OUTPUT` como una tabla de `VARCHAR2(255)`.

```
GET_LINES
GET_LINES(lines OUT CHARARR, numlines IN OUT INTEGER);
```

Ejercicio resuelto

Debes crear un procedimiento que visualice todos los agentes, su nombre, nombre de la familia y/o nombre de la oficina a la que pertenece.

Mostrar retroalimentación

```
CREATE OR REPLACE PROCEDURE lista_agentes IS
    CURSOR cAgentes IS SELECT identificador, nombre,familia, oficina FROM ag
    fam familias.nombre%TYPE;
    ofi oficinas.nombre%TYPE;
    num_ag INTEGER := 0;
BEGIN
    DBMS_OUTPUT.ENABLE( 1000000 );
    DBMS_OUTPUT.PUT_LINE('Agente          |Familia          |Oficina
    DBMS_OUTPUT.PUT_LINE('-----
    FOR ag_rec IN cAgentes LOOP
        IF (ag_rec.familia IS NOT NULL) THEN
            SELECT nombre INTO fam FROM familias WHERE identificador = ag_
            ofi := NULL;
            DBMS_OUTPUT.PUT_LINE(rpad(ag_rec.nombre,20) || '|' || rpad(fam
            num_ag := num_ag + 1;
        ELSIF (ag_rec.oficina IS NOT NULL) THEN
            SELECT nombre INTO ofi FROM oficinas WHERE identificador = ag_
            fam := NULL;
            DBMS_OUTPUT.PUT_LINE(rpad(ag_rec.nombre,20) || '|' || rpad(fam
            num_ag := num_ag + 1;
        END IF;
    END LOOP;
    DBMS_OUTPUT.PUT_LINE('-----
    DBMS_OUTPUT.PUT_LINE('Número de agentes: ' || num_ag);
```

```
END lista_agentes;  
/
```

Recuerda que para ejecutarlo desde SQL*Plus debes ejecutar las siguientes sentencias:

```
SQL>SET SERVEROUTPUT ON;  
SQL>EXEC lista_agentes;
```

4.3.- Objetos.

Hoy día, la programación orientada a objetos es uno de los paradigmas más utilizados y casi todos los lenguajes de programación la soportan. En este apartado vamos a dar unas pequeñas pinceladas de su uso en PL/SQL que serán ampliados en la siguiente unidad de trabajo.

Un tipo de objeto es un tipo de dato compuesto, que encapsula unos datos y las funciones y procedimientos necesarios para manipular esos datos. Las variables son los atributos y los subprogramas son llamados métodos. Podemos pensar en un tipo de objeto como en una entidad que posee unos atributos y un comportamiento (que viene dado por los métodos).

- ✓ Cuando creamos un tipo de objeto, lo que estamos creando es una entidad abstracta que especifica los atributos que tendrán los objetos de ese tipo y define su comportamiento.
- ✓ Cuando instanciamos un objeto estamos particularizando la entidad abstracta a una en particular, con los atributos que tiene el tipo de objeto, pero con un valor dado y con el mismo comportamiento.

Los tipos de objetos tiene 2 partes: una **especificación** y un **cuerpo**. La parte de especificación declara los atributos y los métodos que harán de interfaz de nuestro tipo de objeto. En el cuerpo se implementa la parte de especificación. En la parte de especificación debemos declarar primero los atributos y después los métodos. Todos los atributos son públicos (visibles). No podemos declarar atributos en el cuerpo, pero sí podemos declarar subprogramas locales que serán visibles en el cuerpo del objeto y que nos ayudarán a implementar nuestros métodos.

Los atributos pueden ser de cualquier tipo de datos Oracle, excepto:

- ✓ **LONG** y **LONG RAW**.
- ✓ **NCHAR**, **NCLOB** y **NVARCHAR2**.
- ✓ **MLSLABEL** y **ROWID**.
- ✓ Tipos específicos de PL/SQL: **BINARY_INTEGER**, **BOOLEAN**, **PLS_INTEGER**, **RECORD**, **REF CURSOR**, **%TYPE** y **%ROWTYPE**.
- ✓ Tipos definidos dentro de un paquete PL/SQL.

No podemos inicializar un atributo en la declaración. Tampoco podemos imponerle la restricción **NOT NULL**.

Un **método** es un subprograma declarado en la parte de especificación de un tipo de objeto por medio de: **MEMBER**. Un método no puede llamarse igual que el tipo de objeto o que cualquier atributo. Para cada método en la parte de especificación, debe haber un método implementado en el cuerpo con la misma cabecera.

Todos los métodos en un tipo de objeto aceptan como primer parámetro una instancia de su tipo. Este parámetro es **SELF** y siempre está accesible a un método. Si lo declaramos explícitamente debe ser el primer parámetro, con el nombre **SELF** y del tipo del tipo de objeto. Si **SELF** no está declarado explícitamente, por defecto será **IN** para las funciones e **IN OUT** para los procedimientos.

Los métodos dentro de un tipo de objeto pueden sobrecargarse. No podemos sobrecargarlos si los parámetros formales sólo difieren en el modo o pertenecen a la misma familia. Tampoco podremos sobrecargar una función miembro si sólo difiere en el tipo devuelto.

Una vez que tenemos creado el objeto, podemos usarlo en cualquier declaración. Un objeto cuando se declara sigue las mismas reglas de alcance y visibilidad que cualquier otra variable.

Cuando un objeto se declara éste es automáticamente **NULL**. Dejará de ser nulo cuando lo inicialicemos por medio de su constructor o cuando le asignemos otro. Si intentamos acceder a los atributos de un objeto **NULL** saltará la excepción **ACCES_INT0_NULL**.

Todos los objetos tienen constructores por defecto con el mismo nombre que el tipo de objeto y acepta tantos parámetros como atributos del tipo de objeto y con el mismo tipo. PL/SQL no llama implícitamente a los constructores, deberemos hacerlo nosotros explícitamente.

```
DECLARE
    familia1 Familia;
BEGIN
    ...
    familia1 := Familia( 10, 'Fam10', 1, NULL );
    ...
END;
```

Un tipo de objeto puede tener a otro tipo de objeto entre sus atributos. El tipo de objeto que hace de atributo debe ser conocido por Oracle. Si 2 tipos de objetos son mutuamente dependientes, podemos usar una declaración hacia delante para evitar errores de compilación.

Ejercicio resuelto

Cómo declararías los objetos para nuestra base de datos de ejemplo.

Mostrar retroalimentación

```
CREATE OBJECT Oficina;      --Definición hacia delante
CREATE OBJECT Familia AS OBJECT (
    identificador    NUMBER,
    nombre            VARCHAR2(20),
    familia_         Familia,
    oficina_         Oficina,
    ...
);
CREATE OBJECT Agente AS OBJECT (
    identificador    NUMBER,
    nombre            VARCHAR2(20),
    familia_         Familia,
    oficina_         Oficina,
    ...
);
CREATE OBJECT Oficina AS OBJECT (
    identificador    NUMBER,
    nombre            VARCHAR2(20),
    jefe             Agente,
    ...
);
```

4.3.1.- Objetos. Funciones mapa y funciones de orden.

En la mayoría de los problemas es necesario hacer comparaciones entre tipos de datos, ordenarlos, etc. Sin embargo, los tipos de objetos no tienen orden predefinido por lo que no podrán ser comparados ni ordenados ($x > y$, **DISTINCT**, **ORDER BY**, ...). Nosotros podemos definir el orden que seguirá un tipo de objeto por medio de las funciones mapa y las funciones de orden.

Una función miembro mapa es una función sin parámetros que devuelve un tipo de dato: **DATE**, **NUMBER** o **VARCHAR2** y sería similar a una función hash. Se definen anteponiendo la palabra clave **MAP** y sólo puede haber una para el tipo de objeto.

```
CREATE TYPE Familia AS OBJECT (  
    identificador      NUMBER,  
    nombre             VARCHAR2(20),  
    familia_          NUMBER,  
    oficina_          NUMBER,  
    MAP MEMBER FUNCTION orden RETURN NUMBER,  
    ...  
);  
  
CREATE TYPE BODY Familia AS  
    MAP MEMBER FUNCTION orden RETURN NUMBER IS  
BEGIN  
    RETURN identificador;  
END;  
...  
END;
```

Una función miembro de orden es una función que acepta un parámetro del mismo tipo del tipo de objeto y que devuelve un número negativo si el objeto pasado es menor, cero si son iguales y un número positivo si el objeto pasado es mayor.

```
CREATE TYPE Oficina AS OBJECT (  
    identificador      NUMBER,  
    nombre             VARCHAR2(20),  
    ...  
    ORDER MEMBER FUNCTION igual ( ofi Oficina ) RETURN INTEGER,  
    ...  
);  
  
CREATE TYPE BODY Oficina AS  
    ORDER MEMBER FUNCTION igual ( ofi Oficina ) RETURN INTEGER IS  
BEGIN  
    IF (identificador < ofi.identificador) THEN  
        RETURN -1;  
    ELSIF (identificador = ofi.identificador) THEN  
        RETURN 0;  
    ELSE  
        RETURN 1;  
    END IF;  
END;
```

Autoevaluación

Los métodos de un objeto sólo pueden ser procedimientos.

- ☐ Verdadero.
- ☐ Falso.

Incorrecto, creo que deberías dar un repaso a este apartado.

Efectivamente, ya que las funciones también pueden ser métodos.

Solución

1. Incorrecto
2. Opción correcta

El orden de un objeto se consigue:

- ☐ Al crearlo.
- ☐ En PL/SQL los objetos no pueden ser ordenados.
- ☐ Mediante las funciones mapa y las funciones de orden.

No, los objetos no tienen ningún orden predefinido.

No es cierto, el orden lo podemos conseguir mediante el uso de funciones mapa y funciones de orden.

¡Muy bien!

Solución

1. Incorrecto
2. Incorrecto
3. Opción correcta

5.- Disparadores.

Caso práctico

Juan y María ya han hecho un paquete en el que tienen agrupadas las funciones más usuales que realizan sobre la base de datos de juegos on-line. Sin embargo, **María** ve que hay cosas que aún no pueden controlar. Por ejemplo, **María** quiere que la clave y el usuario de un jugador o jugadora no puedan ser la misma y eso no sabe cómo hacerlo. **Juan** le dice que para ese cometido están los disparadores y sin más dilaciones se pone a explicarle a **María** para qué sirven y cómo utilizarlos.



[ITE \(Loren\)](#) (Uso educativo nc)

En este apartado vamos a tratar una herramienta muy potente e importante proporcionada por PL/SQL para programar nuestra base de datos y mantener la integridad y seguridad que son los disparadores o triggers en inglés.

Pero, ¿qué es un disparador?

Un **disparador** no es más que un procedimiento que es ejecutado cuando se realiza alguna sentencia sobre la BD, o sus tablas, y bajo unas circunstancias establecidas a la hora de definirlo.

Los disparadores pueden ser de tres tipos:

- ✓ **De tablas**
- ✓ **De sustitución**
- ✓ **De sistema**

Un disparador puede ser lanzado antes o después de realizar la operación que lo lanza. Por lo que tendremos disparadores **BEFORE** y disparadores **AFTER**.

Disparadores de Tablas

Normalmente previenen transacciones erróneas o nos permiten implementar restricciones de integridad o seguridad, o automatizar procesos. Son los más utilizados.

Se ejecutan cuando se realiza alguna sentencia de manipulación de datos sobre una tabla dada. Puede ser lanzado al insertar, al actualizar o al borrar de una tabla, por lo que tendremos disparadores **INSERT**, **UPDATE** o **DELETE** (o mezclados).

Puede ser lanzado una vez por sentencia (**FOR STATEMENT**) o una vez por cada fila a la que afecta (**FOR EACH ROW**). Por lo que tendremos disparadores de sentencia y disparadores de fila.

De sustitución

No se ejecutan ni antes ni después, sino en lugar de (se conocen como triggers **INSTEAD OF**). Sólo se pueden asociar a vistas y a nivel de fila.

De sistema

Se ejecutan cuando se produce una determinada operación sobre la BD, se crea una tabla, se conecta un usuario, etc. Los eventos que se pueden detectar son por ejemplo: LOGON, LOGOFF, CREATE, DROP, etc y el momento puede ser tanto **BEFORE** como **AFTER**.

Un **disparador** no es más que un procedimiento y puede ser usado para:

- ✓ Llevar a cabo auditorías sobre la historia de los datos en nuestra base de datos.
- ✓ Garantizar complejas reglas de integridad.
- ✓ Automatizar la generación de valores derivados de columnas.
- ✓ Etc.

Cuando diseñamos un disparador debemos tener en cuenta que:

- ✓ No debemos definir disparadores que dupliquen la funcionalidad que ya incorpora Oracle.
- ✓ Debemos limitar el tamaño de nuestros disparadores, y si estos son muy grandes codificarlos por medio de subprogramas que sean llamados desde el disparador.
- ✓ Cuidar la creación de disparadores recursivos.

Autoevaluación

En PL/SQL sólo podemos definir disparadores de fila.

- ☐ Verdadero.
- ☐ Falso.

No, también podemos definir disparadores de sentencia.

Efectivamente, ya que también podemos definir disparador de sentencia.

Solución

1. Incorrecto
2. Opción correcta

La diferente entre un disparador de fila y uno de sentencia es que el de fila es lanzado una vez por fila a la que afecta la sentencia y el de sentencia es lanzado una sola vez.

- ☐ Verdadero.
- ☐ Falso.

Muy bien, vas por buen camino.

Incorrecto, creo que deberías repasar este apartado otra vez.

Solución

1. Opción correcta
2. Incorrecto

5.1.- Definición de disparadores de Tablas

De los tres tipos de disparadores, veremos los asociados a tablas.

Por lo visto anteriormente, para definir un disparador deberemos indicar si será lanzado antes o después de la ejecución de la sentencia que lo lanza, si se lanzará una vez por sentencia o una vez por fila a la que afecta, y si será lanzado al insertar y/o al actualizar y/o al borrar.

La sintaxis que seguiremos para definir un disparador será la siguiente:

```
CREATE [OR REPLACE] TRIGGER nombre
momento acontecimiento ON tabla
[[REFERENCING (old AS alias_old|new AS alias_new)
FOR EACH ROW
[WHEN condicion]]
bloque_PL/SQL;
```



[ITE \(Loren\)](#) (Uso educativo nc)

Donde nombre nos indica el nombre que le damos al disparador, momento nos dice cuando será lanzado el disparador (**BEFORE** o **AFTER**), acontecimiento será la acción que provoca el lanzamiento del disparador (**INSERT** y/o **DELETE** y/o **UPDATE**). **REFERENCING** y **WHEN** sólo podrán ser utilizados con disparadores para filas. **REFERENCING** nos permite asignar un alias a los valores **NEW** o/y **OLD** de las filas afectadas por la operación, y **WHEN** nos permite indicar al disparador que sólo sea lanzado cuando sea **TRUE** una cierta condición evaluada para cada fila afectada.

En un disparador de fila, podemos acceder a los valores antiguos y nuevos de la fila afectada por la operación, referenciados como **:old** y **:new** (de ahí que podamos utilizar la opción **REFERENCING** para asignar un alias). Si el disparador es lanzado al insertar, el valor antiguo no tendrá sentido y el valor nuevo será la fila que estamos insertando. Para un disparador lanzado al actualizar el valor antiguo contendrá la fila antes de actualizar y el valor nuevo contendrá la fila que vamos actualizar. Para un disparador lanzado al borrar sólo tendrá sentido el valor antiguo.

En el cuerpo de un disparador también podemos acceder a unos predicados que nos dicen qué tipo de operación se está llevando a cabo, que son: **INSERTING**, **UPDATING** y **DELETING**.

Un disparador de fila no puede acceder a la tabla asociada. Se dice que esa tabla está mutando. Si un disparador es lanzado en cascada por otro disparador, éste no podrá acceder a ninguna de las tablas asociadas, y así recursivamente.

```
CREATE TRIGGER prueba BEFORE UPDATE ON agentes
FOR EACH ROW
BEGIN
    ...
    SELECT identificador FROM agentes WHERE ...
/*devolvería el error ORA-04091: table AGENTES is mutating, trigger/function may not see it*/
    ...
END;
/
```

Si tenemos varios tipos de disparadores sobre una misma tabla, el orden de ejecución será:

- ✓ Triggers before de sentencia.
- ✓ Triggers before de fila.
- ✓ Triggers after de fila.
- ✓ Triggers after de sentencia.

Existe una vista del diccionario de datos con información sobre los disparadores: **USER_TRIGGERS**;

```
SQL>DESC USER_TRIGGERS;
```

Name	Null?	Type
-----	-----	----
TRIGGER_NAME	NOT NULL	VARCHAR2(30)
TRIGGER_TYPE		VARCHAR2(16)
TRIGGERING_EVENT		VARCHAR2(26)
TABLE_OWNER	NOT NULL	VARCHAR2(30)
TABLE_NAME	NOT NULL	VARCHAR2(30)
REFERENCING_NAMES		VARCHAR2(87)
WHEN_CLAUSE		VARCHAR2(4000)
STATUS		VARCHAR2(8)
DESCRIPTION		VARCHAR2(4000)
TRIGGER_BODY		LONG

5.2.- Ejemplos de disparadores.

Ya hemos visto qué son los disparadores, los tipos que existen, cómo definirlos y algunas consideraciones a tener en cuenta a la hora de trabajar con ellos.

Ahora vamos a ver algunos ejemplos de su utilización con los que podremos comprobar la potencia que éstos nos ofrecen.

Ejercicio resuelto

Como un agente debe pertenecer a una familia o una oficina pero no puede pertenecer a una familia y a una oficina a la vez, deberemos implementar un disparador para llevar a cabo esta restricción que Oracle no nos permite definir desde el DDL.

Mostrar retroalimentación

Para este cometido definiremos un disparador de fila que saltará antes de que insertemos o actualicemos una fila en la tabla agentes, cuyo código podría ser el siguiente:

```
CREATE OR REPLACE TRIGGER integridad_agentes
BEFORE INSERT OR UPDATE ON agentes
FOR EACH ROW
BEGIN
    IF (:new.familia IS NULL and :new.oficina IS NULL) THEN -- Si los dos va
        RAISE_APPLICATION_ERROR(-20201, 'Un agente no puede ser huérfano');
    ELSIF (:new.familia IS NOT NULL and :new.oficina IS NOT NULL) THEN -- Si
        RAISE_APPLICATION_ERROR(-20202, 'Un agente no puede tener dos padre
    END IF;
END;
/
```

Debes conocer

En la siguiente presentación podrás ver la descripción de la tabla `USER_TRIGGERS`, cómo se almacena un disparador en la base de datos, cómo se consulta su código y cómo se lanza su ejecución. Pruébalo en tu máquina para comprobar su funcionamiento.

Ejercicio resuelto

Supongamos que tenemos una tabla de históricos para agentes que nos permita auditar las familias y oficinas por la que ha ido pasando un agente. La tabla tiene la fecha de inicio y la fecha de finalización del agente en esa familia u oficina, el identificador del agente, el nombre del agente, el nombre de la familia y el nombre de la oficina. Queremos hacer un disparador que inserte en esa tabla.

Mostrar retroalimentación

Para llevar a cabo esta tarea definiremos un disparador de fila que saltará después de insertar, actualizar o borrar una fila en la tabla agentes, cuyo código podría ser el siguiente:

```
CREATE OR REPLACE TRIGGER historico_agentes
AFTER INSERT OR UPDATE OR DELETE ON agentes
FOR EACH ROW
DECLARE
    oficina VARCHAR2(40);
    familia VARCHAR2(40);
    ahora DATE := sysdate;
BEGIN
    IF INSERTING THEN
        IF (:new.familia IS NOT NULL) THEN
            SELECT nombre INTO familia FROM familias WHERE identificador =
            oficina := NULL;
        ELSIF
            SELECT nombre INTO oficina FROM oficinas WHERE identificador =
            familia := NULL;
        END IF;
        INSERT INTO histagentes VALUES (ahora, NULL, :new.identificador, :n
        COMMIT;
    ELSIF UPDATING THEN
        UPDATE histagentes SET fecha_hasta = ahora WHERE identificador = :o
        IF (:new.familia IS NOT NULL) THEN
            SELECT nombre INTO familia FROM familias WHERE identificador =
            oficina := NULL;
        ELSE
            SELECT nombre INTO oficina FROM oficinas WHERE identificador =
            familia := NULL;
        END IF;
        INSERT INTO histagentes VALUES (ahora, NULL, :new.identificador, :n
        COMMIT;
    ELSE
        UPDATE histagentes SET fecha_hasta = ahora WHERE identificador = :o
        COMMIT;
```

```
END IF;  
END;  
/
```

Ejercicio resuelto

Queremos realizar un disparador que no nos permita llevar a cabo operaciones con familias si no estamos en la jornada laboral.

Mostrar retroalimentación

```
CREATE OR REPLACE TRIGGER jornada_familias  
BEFORE INSERT OR DELETE OR UPDATE ON familias  
DECLARE  
    ahora DATE := sysdate;  
BEGIN  
    IF (TO_CHAR(ahora, 'DY') = 'SAT' OR TO_CHAR(ahora, 'DY') = 'SUN') THEN  
        RAISE_APPLICATION_ERROR(-20301, 'No podemos manipular familias en f  
    END IF;  
    IF (TO_CHAR(ahora, 'HH24') < 8 OR TO_CHAR(ahora, 'HH24') > 18) THEN  
        RAISE_APPLICATION_ERROR(-20302, 'No podemos manipular familias fuer  
    END IF;  
END;  
/
```

6.- Interfaces de programación de aplicaciones para lenguajes externos.

Caso práctico

Juan y María tienen la base de datos de juegos on-line lista, pero no tienen claro si todo lo que han hecho lo tienen que utilizar desde dentro de la base de datos o si pueden reutilizar todo ese trabajo desde otro lenguaje de programación desde el que están creando la interfaz web desde la que los jugadores y jugadoras se conectarán para jugar y desde la que también se conectará la administradora para gestionar la base de datos de una forma más amigable.

En ese momento pasa **Ada** al lado y ambos la asaltan con su gran duda. **Ada**, muy segura, les responde que para eso existen las interfaces de programación de aplicaciones o APIs, que les permiten acceder desde otros lenguajes de programación a la base de datos.



[ITE \(Abraham Pérez Pérez\)](#) (Uso educativo nc)

Llegados a este punto ya sabemos programar nuestra base de datos, pero al igual que a **Juan y María** te surgirá la duda de si todo lo que has hecho puedes aprovecharlo desde cualquier otro lenguaje de programación. La respuesta es la misma que dio **Ada**: Sí.

En este apartado te vamos a dar algunas referencias sobre las APIs para acceder a las bases de datos desde un lenguaje de programación externo. No vamos a ser exhaustivos ya que eso queda fuera del alcance de esta unidad e incluso de este módulo. Todo ello lo vas a estudiar con mucha más profundidad en otros módulos de este ciclo (o incluso ya lo conoces). Aquí sólo pretendemos que sepas que existen y que las puedes utilizar.

Las primeras APIs utilizadas para acceder a bases de datos Oracle fueron las que el mismo Oracle proporcionaba: Pro*C, Pro*Fortran y Pro*Cobol. Todas permitían embeber llamadas a la base de datos en nuestro programa y a la hora de compilarlo, primero debíamos pasar el precompilador adecuado que trasladaba esas llamadas embebidas en llamadas a una librería utilizada en tiempo de ejecución. Sin duda, el más utilizado fue Pro*C, ya que el lenguaje C y C++ tuvieron un gran auge.

Para saber más

En los siguientes enlaces podrás obtener más información sobre Pro*C y cómo crear un programa para acceder a nuestra base de datos.

[Introducción a Pro*C.](#)

[Ejemplo de programa en Pro*C.](#)

Hoy día existen muchas más APIs para acceder a las bases de datos ya que tanto los lenguajes de programación como las tecnologías han evolucionado mucho. Antes la programación para la web casi no existía y por eso todos los programas que accedían a bases de datos lo hacían bien en local o bien a través de una red local. Hoy día eso sería impensable, de ahí que las APIs también hayan evolucionado mucho y tengamos una gran oferta a nuestro alcance para elegir la que más se adecue a nuestras necesidades.

Para saber más

En los siguientes enlaces podrás ampliar información sobre algunas APIs muy comunes para acceder a bases de datos.

[Tecnologías Java para acceder a bases de datos.](#)

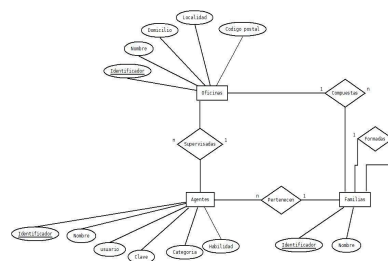
[Conectividad abierta de bases de datos. \(ODBC\)](#)

Anexo I.- Caso de estudio.

Una empresa de telefonía tiene sus centros de llamadas distribuidos por la geografía española en diferentes oficinas. Estas oficinas están jerarquizadas en familias de agentes telefónicos. Cada familia, por tanto, podrá contener agentes u otras familias. Los agentes telefónicos, según su categoría, además se encargarán de supervisar el trabajo de todos los agentes de una oficina o de coordinar el trabajo de los agentes de una familia dada. El único agente que pertenecerá directamente a una oficina y que no formará parte de ninguna familia será el supervisor de dicha oficina, cuya categoría es la 2. Los coordinadores de las familias deben pertenecer a dicha familia y su categoría será 1 (no todas las familias tienen por qué tener un coordinador y dependerá del tamaño de la oficina, ya que de ese trabajo también se puede encargar el supervisor de la oficina). Los demás agentes deberán pertenecer a una familia, su categoría será 0 y serán los que principalmente se ocupen de atender las llamadas.

- ✓ De los agentes queremos conocer su nombre, su clave y contraseña para entrar al sistema, su categoría y su habilidad que será un número entre 0 y 9 indicando su habilidad para atender llamadas.
- ✓ Para las familias sólo nos interesa conocer su nombre.
- ✓ Finalmente, para las oficinas queremos saber su nombre, domicilio, localidad y código postal de la misma.

Un posible modelo entidad-relación para el problema expuesto podría ser el siguiente:



[Ministerio de Educación](#) (Uso educativo nc)

El **Modelo Relacional** resultante sería:

OFICINAS (identificador, nombre, domicilio, localidad, codigo_postal)

FAMILIAS (identificador, nombre, familia (fk), oficina (fk))

AGENTES (identificador, nombre, usuario, clave, habilidad, categoría, familia (fk), oficina (fk))

De este modelo de datos surgen tres tablas, que puedes crear en Oracle con el script del siguiente enlace

[Script CreaCasoEstudio.zip](#) (zip - 1,62 KB)

El script crea un usuario llamado c##agencia con clave agencia. Para que puedas ejecutarlo y comenzar de cero, cuantas veces quieras, el script elimina el usuario c##agencia y sus tablas antes de volver a crearlo.

Conecta como administrador con SYS as SYSDBA y ejecútalo anteponiendo el símbolo @ o la palabra start antes del nombre del script.

```
C:\app\admin\product\18.0.0\dbhomeXE\bin\sqlplus.exe

SQL*Plus: Release 18.0.0.0.0 - Production on Mié Ago 19 10:59:05 2020
Version 18.4.0.0.0

Copyright (c) 1982, 2018, Oracle. All rights reserved.

Introduzca el nombre de usuario: sys as sysdba
Introduzca la contraseña:

Conectado a:
Oracle Database 18c Express Edition Release 18.0.0.0.0 - Production
Version 18.4.0.0.0

SQL> start C:\CreaCasoEstudio.sql

Usuario borrado.

Usuario creado.

Concesión terminada correctamente.

Conectado.

Tabla creada.

Tabla creada.

Tabla creada.

1 fila creada.
```

[Oracle Corporation](#) (Todos los derechos reservados)

Habilitando la Salida/OUTPUT en Bloques PL/SQL.

PL/SQL no proporciona funcionalidad de entrada o salida directamente siendo necesario utilizar paquetes predefinidos de **Oracle** para tales fines. Para generar una salida debes realizar lo siguiente :

1. Ejecutar el siguiente comando tanto en **SQL*Plus** como en cualquier otro entorno

SET SERVEROUTPUT ON

2. En el bloque **PL/SQL**, hay que utilizar el procedimiento **PUT_LINE** del paquete **DBMS_OUTPUT** para mostrar la salida. El valor a mostrar en pantalla se pasará como argumento del procedimiento.

Ejecuta el siguiente código desde SQLPlus y desde SQLDeveloper para comprobar su funcionamiento.

```
SET SERVEROUTPUT ON
BEGIN
    DBMS_OUTPUT.PUT_LINE('Hola mundo');
END;
```

Anexo II.- Excepciones predefinidas en Oracle.

Las excepciones predefinidas son:

Excepciones predefinidas en Oracle.

Excepción.	SQLCODE	Lanzada cuando ...
ACCES_INT0_NULL	-6530	Intentamos asignar valor a atributos de objetos no inicializados.
COLECTION_IS_NULL	-6531	Intentamos asignar valor a elementos de colecciones no inicializadas, o acceder a métodos distintos de EXISTS.
CURSOR_ALREADY_OPEN	-6511	Intentamos abrir un cursor ya abierto.
DUP_VAL_ON_INDEX	-1	Índice único violado.
INVALID_CURSOR	-1001	Intentamos hacer una operación con un cursor que no está abierto.
INVALID_NUMBER	-1722	Conversión de cadena a número falla.
LOGIN_DENIED	-1403	El usuario y/o contraseña para conectarnos a Oracle no es válido.
NO_DATA_FOUND	+100	Una sentencia SELECT no devuelve valores, o intentamos acceder a un elemento borrado de una tabla anidada.
NOT_LOGGED_ON	-1012	No estamos conectados a Oracle.
PROGRAM_ERROR	-6501	Ha ocurrido un error interno en PL/SQL.
ROWTYPE_MISMATCH	-6504	Diferentes tipos en la asignación de 2 cursores.
STORAGE_ERROR	-6500	Memoria corrupta.
SUBSCRIPT_BEYOND_COUNT	-6533	El índice al que intentamos acceder en una colección sobrepasa su límite superior.
SUBSCRIPT_OUTSIDE_LIMIT	-6532	Intentamos acceder a un rango no válido dentro de una colección (-1 por ejemplo).
TIMEOUT_ON_RESOURCE	-51	Un timeout ocurre mientras Oracle espera por un recurso.
TOO_MANY_ROWS	-1422	Una sentencia SELECT...INT0... devuelve más de una fila.
VALUE_ERROR	-6502	Ocurre un error de conversión, aritmético, de truncado o de restricción de tamaño.

Excepción.	SQLCODE	Lanzada cuando ...
ZERO_DIVIDE	-1476	Intentamos dividir un número por 0.

Anexo III.- Evaluación de los atributos de un cursor explícito.

Evaluación de los atributos de un cursor explícito según las operaciones realizadas con él.

Operación realizada.	%FOUND	%NOTFOUND	%ISOPEN	%ROWCOUNT
Antes del OPEN	Excepción.	Excepción.	FALSE	Excepción.
Después del OPEN	NULL	NULL	TRUE	0
Antes del primer FETCH	NULL	NULL	TRUE	0
Después del primer FETCH	TRUE	FALSE	TRUE	1
Antes de los siguientes FETCH	TRUE	FALSE	TRUE	1
Después de los siguientes FETCH	TRUE	FALSE	TRUE	Depende datos.
Antes del último FETCH	TRUE	FALSE	TRUE	Depende datos.
Después del último FETCH	FALSE	TRUE	TRUE	Depende datos.
Antes del CLOSE	FALSE	TRUE	TRUE	Depende datos.
Después del CLOSE	Excepción.	Excepción.	FALSE	Excepción.

Anexo IV.- Paso de parámetros a subprogramas.

Veamos algunos ejemplos del paso de parámetros a subprogramas:

✓ Notación mixta.

```
DECLARE
    PROCEDURE prueba( formal1 NUMBER, formal2 VARCHAR2) IS
    BEGIN
        ...
    END;
    actual1 NUMBER;
    actual2 VARCHAR2;
BEGIN
    ...
    prueba(actual1, actual2);           --posicional
    prueba(formal2=>actual2,formal1=>actual1);    --nombrada
    prueba(actual1, formal2=>actual2);    --mixta
END;
```

✓ Parámetros de entrada.

```
FUNCTION categoria( id_agente IN NUMBER )
RETURN NUMBER IS
    cat NUMBER;
BEGIN
    ...
    SELECT categoria INTO cat FROM agentes
WHERE identificador = id_agente;
RETURN cat;
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        id_agente := -1; --ilegal, parámetro de entrada
END;
```

✓ Parámetros de salida.

```
PROCEDURE nombre( id_agente NUMBER, nombre OUT VARCHAR2) IS
BEGIN
    IF (nombre = 'LUIS') THEN    --error de sintaxis
    END IF;
    ...
END;
```

✔ Parámetros con valor por defecto de los que podemos prescindir.

```
DECLARE
    SUBTYPE familia IS familias%ROWTYPE;
    SUBTYPE agente IS agentes%ROWTYPE;
    SUBTYPE tabla_agentes IS TABLE OF agente;
    familia1 familia;
    familia2 familia;
    hijos_fam tabla_agentes;
    FUNCTION inserta_familia( mi_familia familia,
        mis_agentes tabla_agentes := tabla_agentes() )
RETURN NUMBER IS
    BEGIN
        INSERT INTO familias VALUES (mi_familia);
        FOR i IN 1..mis_agentes.COUNT LOOP
            IF (mis_agentes(i).oficina IS NOT NULL) or (mis_agentes(i).familia != mi_fam
                ROLLBACK;
                RETURN -1;
            END IF;
            INSERT INTO agentes VALUES (mis_agentes(i));
        END LOOP;
        COMMIT;
        RETURN 0;
    EXCEPTION
        WHEN DUP_VAL_ON_INDEX THEN
            ROLLBACK;
            RETURN -1;
        WHEN OTHERS THEN
            ROLLBACK;
            RETURN SQLCODE;
    END inserta_familia;
BEGIN
    ...
    resultado := inserta_familia( familia1 );
    ...
    resultado := inserta_familia( familia2, hijos_fam2 );
    ...
END;
```


Anexo V.- Sobrecarga de subprogramas.

Veamos un ejemplo de sobrecarga de subprogramas en el que una misma función es sobrecargada tres veces para diferentes tipos de parámetros.

```
DECLARE
    TYPE agente IS agentes%ROWTYPE;
    TYPE familia IS familias%ROWTYPE;
    TYPE tAgentes IS TABLE OF agente;
    TYPE tFamilias IS TABLE OF familia;

    FUNCTION inserta_familia( mi_familia familia )
    RETURN NUMBER IS
    BEGIN
        INSERT INTO familias VALUES (mi_familia.identificador, mi_familia.nombre, mi_familia.fam
        COMMIT;
        RETURN 0;
    EXCEPTION
        WHEN DUP_VAL_ON_INDEX THEN
            ROLLBACK;
            RETURN -1;
        WHEN OTHERS THEN
            ROLLBACK;
            RETURN SQLCODE;
    END inserta_familia;

    FUNCTION inserta_familia( mi_familia familia, hijas tFamilias )
    RETURN NUMBER IS
    BEGIN
        INSERT INTO familias VALUES (mi_familia.identificador, mi_familia.nombre, mi_familia.fam
        IF (hijas IS NOT NULL) THEN
            FOR i IN 1..hijas.COUNT LOOP
                IF (hijas(i).oficina IS NOT NULL) or (hijas(i).familia != mi_familia.identifi
                    ROLLBACK;
                    RETURN -1;
                END IF;
                INSERT INTO familias VALUES (hijas(i).identificador, hijas(i).nombre, hijas(i
            END LOOP;
        END IF;
        COMMIT;
        RETURN 0;
    EXCEPTION
        WHEN DUP_VAL_ON_INDEX THEN
            ROLLBACK;
            RETURN -1;
        WHEN OTHERS THEN
            ROLLBACK;
            RETURN -1;
    END inserta_familia;

    FUNCTION inserta_familia( mi_familia familia, hijos tAgentes )
    RETURN NUMBER IS
    BEGIN
        INSERT INTO familias VALUES (mi_familia.identificador, mi_familia.nombre, mi_familia.fam
        IF (hijos IS NOT NULL) THEN
```

```

        FOR i IN 1..hijos.COUNT LOOP
            IF (hijos(i).oficina IS NOT NULL) or (hijos(i).familia != mi_familia.identifi
                ROLLBACK;
                RETURN -1;
            END IF;
            INSERT INTO agentes VALUES (hijos(i).identificador, hijos(i).nombre, hijos(i)
        END LOOP;
    END IF;
    COMMIT;
    RETURN 0;
EXCEPTION
    WHEN DUP_VAL_ON_INDEX THEN
        ROLLBACK;
        RETURN -1;
    WHEN OTHERS THEN
        ROLLBACK;
        RETURN -1;
END inserta_familias;

mi_familia familia;
mi_familia1 familia;
familias_hijas tFamilias;
mi_familia2 familia;
hijos tAgentes;
BEGIN
    ...
    resultado := inserta_familia(mi_familia);
    ...
    resultado := inserta_familia(mi_familia1, familias_hijas);
    ...
    resultado := inserta_familia(mi_familia2, hijos);
    ...
END;
```

Anexo VI.- Ejemplo de recursividad.

Aquí tienes un ejemplo del uso de la recursividad en nuestros subprogramas.

```
DECLARE
    TYPE agente IS agentes%ROWTYPE;
    TYPE tAgentes IS TABLE OF agente;
    hijos10 tAgentes;
    PROCEDURE dame_hijos( id_familia NUMBER,
        hijos IN OUT tAgentes ) IS
        CURSOR hijas IS SELECT identificador FROM familias WHERE familia = id_familia;
    hija NUMBER;
    CURSOR cHijos IS SELECT * FROM agentes WHERE familia = id_familia;
    hijo agente;
    BEGIN
        --Si la tabla no está inicializada -> la inicializamos
        IF hijos IS NULL THEN
            hijos = tAgentes();
        END IF;
        --Metemos en la tabla los hijos directos de esta familia
        OPEN cHijos;
        LOOP
            FETCH cHijos INTO hijo;
            EXIT WHEN cHijos%NOTFOUND;
            hijos.EXTEND;
            hijos(hijos.LAST) := hijo;
        END LOOP;
        CLOSE cHijos;
        --Hacemos lo mismo para todas las familias hijas de la actual
        OPEN hijas;
        LOOP
            FETCH hijas INTO hija;
            EXIT WHEN hijas%NOTFOUND;
            dame_hijos( hija, hijos );
        END LOOP;
        CLOSE hijas;
    END dame_hijos;
BEGIN
    ...
    dame_hijos( 10, hijos10 );
    ...
END;
```

Anexo VII.- Ejemplo de paquete.

Aquí puedes ver un ejemplo de un paquete que agrupa las principales tareas que llevamos a cabo con la base de datos de ejemplo.

```
CREATE OR REPLACE PACKAGE call_center AS      --inicialización
--Definimos los tipos que utilizaremos
SUBTYPE agente IS agentes%ROWTYPE;
SUBTYPE familia IS familias%ROWTYPE;
SUBTYPE oficina IS oficinas%ROWTYPE;
TYPE tAgentes IS TABLE OF agente;
TYPE tFamilias IS TABLE OF familia;
TYPE tOficinas IS TABLE OF oficina;

--Definimos las excepciones propias
referencia_no_encontrada exception;
referencia_encontrada exception;
no_null exception;
PRAGMA EXCEPTION_INIT(referencia_no_encontrada, -2291);
PRAGMA EXCEPTION_INIT(referencia_encontrada, -2292);
PRAGMA EXCEPTION_INIT(no_null, -1400);

--Definimos los errores que vamos a tratar
todo_bien                CONSTANT NUMBER := 0;
elemento_existente        CONSTANT NUMBER:= -1;
elemento_inexistente      CONSTANT NUMBER:= -2;
padre_existente           CONSTANT NUMBER:= -3;
padre_inexistente         CONSTANT NUMBER:= -4;
no_null_violado           CONSTANT NUMBER:= -5;
operacion_no_permitida    CONSTANT NUMBER:= -6;

--Definimos los subprogramas públicos
--Nos devuelve la oficina padre de un agente
PROCEDURE oficina_padre( mi_agente agente, padre OUT oficina );

--Nos devuelve la oficina padre de una familia
PROCEDURE oficina_padre( mi_familia familia, padre OUT oficina );

--Nos da los hijos de una familia
PROCEDURE dame_hijos( mi_familia familia, hijos IN OUT tAgentes );

--Nos da los hijos de una oficina
PROCEDURE dame_hijos( mi_oficina oficina, hijos IN OUT tAgentes );

--Inserta un agente
FUNCTION inserta_agente ( mi_agente agente )
RETURN NUMBER;

--Inserta una familia
FUNCTION inserta_familia( mi_familia familia )
RETURN NUMBER;

--Inserta una oficina
FUNCTION inserta_oficina ( mi_oficina oficina )
RETURN NUMBER;
```

```

--Borramos una oficina
FUNCTION borra_oficina( id_oficina NUMBER )
RETURN NUMBER;

--Borramos una familia
FUNCTION borra_familia( id_familia NUMBER )
RETURN NUMBER;

--Borramos un agente
FUNCTION borra_agente( id_agente NUMBER )
RETURN NUMBER;
END call_center;
/
CREATE OR REPLACE PACKAGE BODY call_center AS          --cuerpo
--Implemento las funciones definidas en la especificación

--Nos devuelve la oficina padre de un agente
PROCEDURE oficina_padre( mi_agente agente, padre OUT oficina ) IS
    mi_familia familia;
BEGIN
    IF (mi_agente.oficina IS NOT NULL) THEN
        SELECT * INTO padre FROM oficinas
        WHERE identificador = mi_agente.oficina;
    ELSE
        SELECT * INTO mi_familia FROM familias
        WHERE identificador = mi_agente.familia;
        oficina_padre( mi_familia, padre );
    END IF;
EXCEPTION
    WHEN OTHERS THEN
        padre := NULL;
END oficina_padre;

--Nos devuelve la oficina padre de una familia
PROCEDURE oficina_padre( mi_familia familia, padre OUT oficina ) IS
    madre familia;
BEGIN
    IF (mi_familia.oficina IS NOT NULL) THEN
        SELECT * INTO padre FROM oficinas
        WHERE identificador = mi_familia.oficina;
    ELSE
        SELECT * INTO madre FROM familias
        WHERE identificador = mi_familia.familia;
        oficina_padre( madre, padre );
    END IF;
EXCEPTION
    WHEN OTHERS THEN
        padre := NULL;
END oficina_padre;

--Nos da los hijos de una familia
PROCEDURE dame_hijos( mi_familia familia, hijos IN OUT tAgentes ) IS
    CURSOR cHijos IS SELECT * FROM agentes
    WHERE familia = mi_familia.identificador;
    CURSOR cHijas IS SELECT * FROM familias
    WHERE familia = mi_familia.identificador;
    hijo agente;
    hija familia;
BEGIN

```

```

--inicializamos la tabla si no lo está
if (hijos IS NULL) THEN
    hijos := tAgentes();
END IF;
--metemos en la tabla los hijos directos
OPEN cHijos;
LOOP
    FETCH cHijos INTO hijo;
    EXIT WHEN cHijos%NOTFOUND;
    hijos.EXTEND;
    hijos(hijos.LAST) := hijo;
END LOOP;
CLOSE cHijos;
--hacemos lo mismo para las familias hijas
OPEN cHijas;
LOOP
    FETCH cHijas INTO hija;
    EXIT WHEN cHijas%NOTFOUND;
    dame_hijos( hija, hijos );
END LOOP;
CLOSE cHijas;
EXCEPTION
    WHEN OTHERS THEN
        hijos := tAgentes();
END dame_hijos;

--Nos da los hijos de una oficina
PROCEDURE dame_hijos( mi_oficina oficina, hijos IN OUT tAgentes ) IS
    CURSOR cHijos IS SELECT * FROM agentes
    WHERE oficina = mi_oficina.identificador;
    CURSOR cHijas IS SELECT * FROM familias
    WHERE oficina = mi_oficina.identificador;
    hijo agente;
    hija familia;
BEGIN
    --inicializamos la tabla si no lo está
    if (hijos IS NULL) THEN
        hijos := tAgentes();
    END IF;
    --metemos en la tabla los hijos directos
    OPEN cHijos;
    LOOP
        FETCH cHijos INTO hijo;
        EXIT WHEN cHijos%NOTFOUND;
        hijos.EXTEND;
        hijos(hijos.LAST) := hijo;
    END LOOP;
    CLOSE cHijos;
    --hacemos lo mismo para las familias hijas
    OPEN cHijas;
    LOOP
        FETCH cHijas INTO hija;
        EXIT WHEN cHijas%NOTFOUND;
        dame_hijos( hija, hijos );
    END LOOP;
    CLOSE cHijas;
EXCEPTION
    WHEN OTHERS THEN
        hijos := tAgentes();
END dame_hijos;

```

```

--Inserta un agente
FUNCTION inserta_agente ( mi_agente agente )
RETURN NUMBER IS
BEGIN
    IF (mi_agente.familia IS NULL and mi_agente.oficina IS NULL) THEN
        RETURN operacion_no_permitida;
    END IF;
    IF (mi_agente.familia IS NOT NULL and mi_agente.oficina IS NOT NULL) THEN
        RETURN operacion_no_permitida;
    END IF;
    INSERT INTO agentes VALUES (mi_agente.identificador, mi_agente.nombre, mi_agente.usuario);
    COMMIT;
    RETURN todo_bien;
EXCEPTION
    WHEN referencia_no_encontrada THEN
        ROLLBACK;
        RETURN padre_inexistente;
    WHEN no_null THEN
        ROLLBACK;
        RETURN no_null_violado;
    WHEN DUP_VAL_ON_INDEX THEN
        ROLLBACK;
        RETURN elemento_existente;
    WHEN OTHERS THEN
        ROLLBACK;
        RETURN SQLCODE;
END inserta_agente;

--Inserta una familia
FUNCTION inserta_familia( mi_familia familia )
RETURN NUMBER IS
BEGIN
    IF (mi_familia.familia IS NULL and mi_familia.oficina IS NULL) THEN
        RETURN operacion_no_permitida;
    END IF;
    IF (mi_familia.familia IS NOT NULL and mi_familia.oficina IS NOT NULL) THEN
        RETURN operacion_no_permitida;
    END IF;
    INSERT INTO familias VALUES ( mi_familia.identificador, mi_familia.nombre, mi_familia.familia );
    COMMIT;
    RETURN todo_bien;
EXCEPTION
    WHEN referencia_no_encontrada THEN
        ROLLBACK;
        RETURN padre_inexistente;
    WHEN no_null THEN
        ROLLBACK;
        RETURN no_null_violado;
    WHEN DUP_VAL_ON_INDEX THEN
        ROLLBACK;
        RETURN elemento_existente;
    WHEN OTHERS THEN
        ROLLBACK;
        RETURN SQLCODE;
END inserta_familia;

--Inserta una oficina
FUNCTION inserta_oficina ( mi_oficina oficina )
RETURN NUMBER IS

```

```

BEGIN
    INSERT INTO oficinas VALUES (mi_oficina.identificador, mi_oficina.nombre, mi_oficina.do
    COMMIT;
    RETURN todo_bien;
EXCEPTION
    WHEN no_null THEN
        ROLLBACK;
        RETURN no_null_violado;
    WHEN DUP_VAL_ON_INDEX THEN
        ROLLBACK;
        RETURN elemento_existente;
    WHEN OTHERS THEN
        ROLLBACK;
        RETURN SQLCODE;
END inserta_oficina;

--Borramos una oficina
FUNCTION borra_oficina( id_oficina NUMBER )
RETURN NUMBER IS
    num_ofi NUMBER;
BEGIN
    SELECT COUNT(*) INTO num_ofi FROM oficinas
    WHERE identificador = id_oficina;
    IF (num_ofi = 0) THEN
        RETURN elemento_inexistente;
    END IF;
    DELETE oficinas WHERE identificador = id_oficina;
    COMMIT;
    RETURN todo_bien;
EXCEPTION
    WHEN OTHERS THEN
        ROLLBACK;
        RETURN SQLCODE;
END borra_oficina;

--Borramos una familia
FUNCTION borra_familia( id_familia NUMBER )
RETURN NUMBER IS
    num_fam NUMBER;
BEGIN
    SELECT COUNT(*) INTO num_fam FROM familias
    WHERE identificador = id_familia;
    IF (num_fam = 0) THEN
        RETURN elemento_inexistente;
    END IF;
    DELETE familias WHERE identificador = id_familia;
    COMMIT;
    RETURN todo_bien;
EXCEPTION
    WHEN OTHERS THEN
        ROLLBACK;
        RETURN SQLCODE;
END borra_familia;

--Borramos un agente
FUNCTION borra_agente( id_agente NUMBER )
RETURN NUMBER IS
    num_ag NUMBER;
BEGIN
    SELECT COUNT(*) INTO num_ag FROM agentes

```



```
WHERE identificador = id_agente;
IF (num_ag = 0) THEN
    RETURN elemento_inexistente;
END IF;
DELETE agentes WHERE identificador = id_agente;
COMMIT;
RETURN todo_bien;
EXCEPTION
    WHEN OTHERS THEN
        ROLLBACK;
        RETURN SQLCODE;
END borra_agente;
END call_center;
/
```